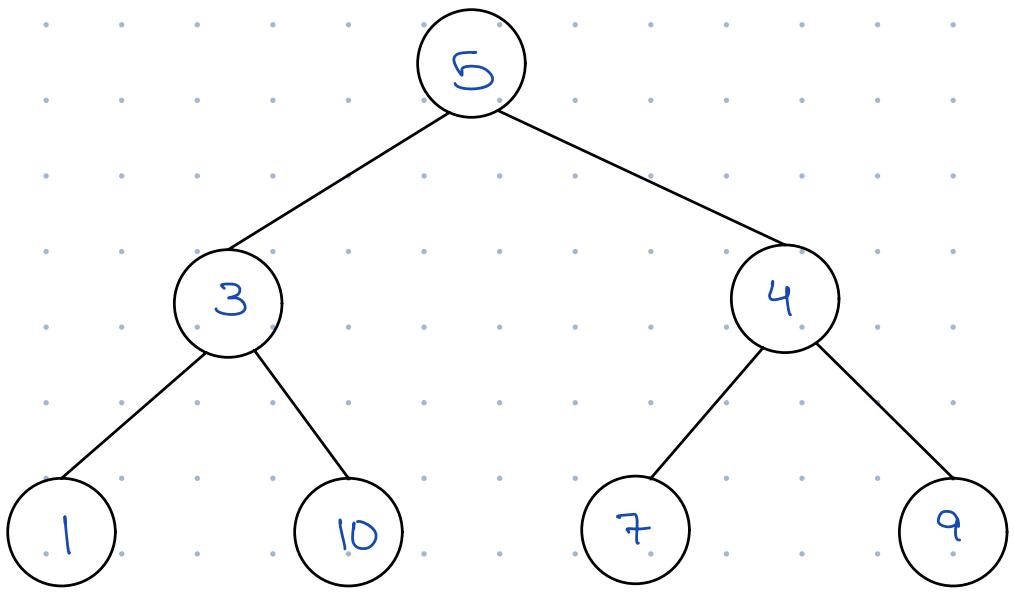
Binary Tree

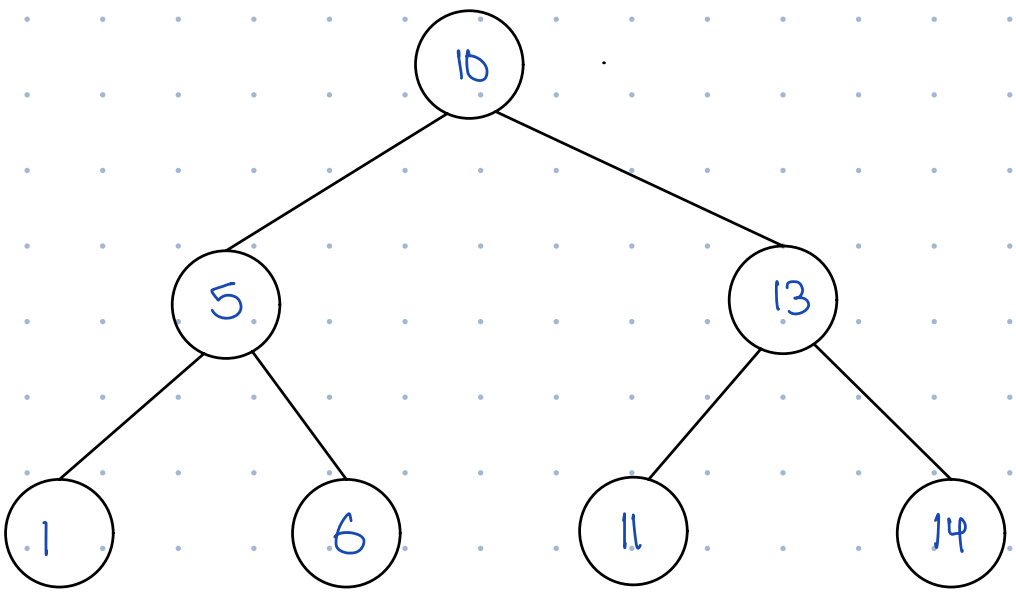


- Levels start from root i.e level 0
- height starts from bottom i.e. leaf nodes at height = 0.

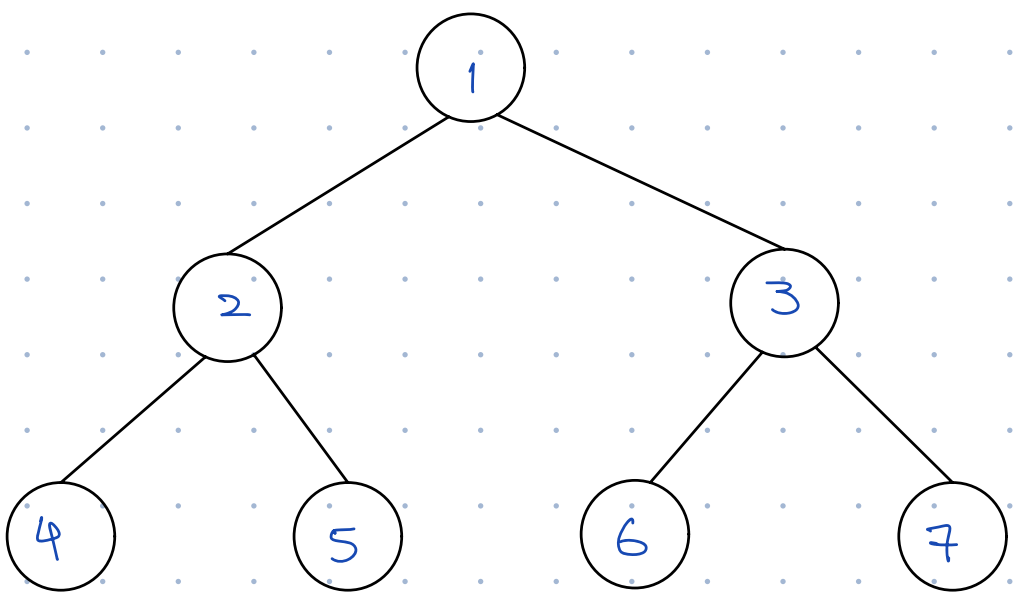
max nodes at height h =

$$\left\lceil \frac{n}{2^{h+1}} \right\rceil$$

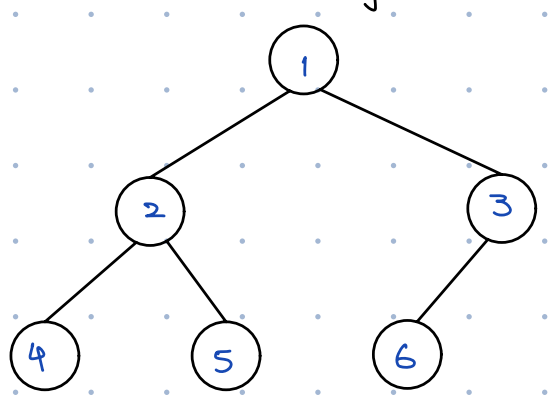
Binary Search Tree



Perfect Binary Tree



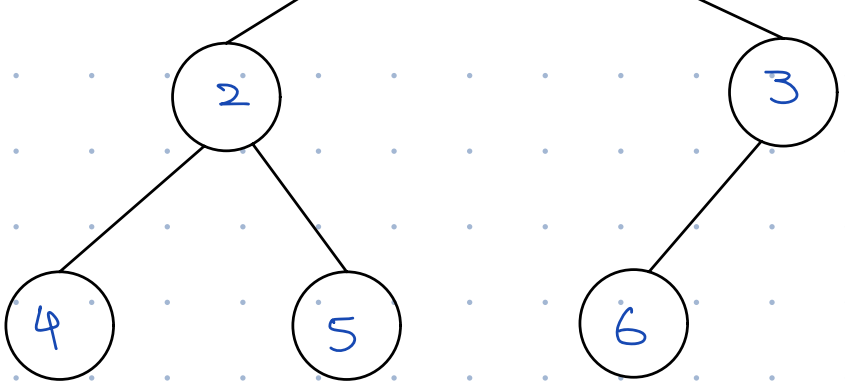
NOT Perfect

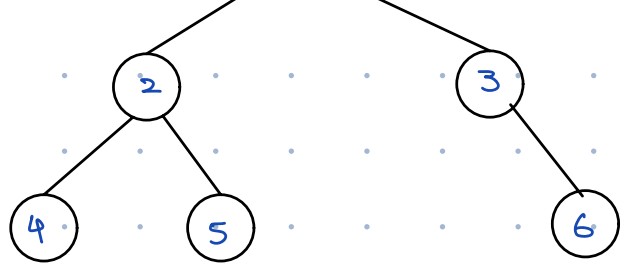


- All leaf nodes are on the same level
- All internal nodes have two children

Complete Binary Tree

NOT complete



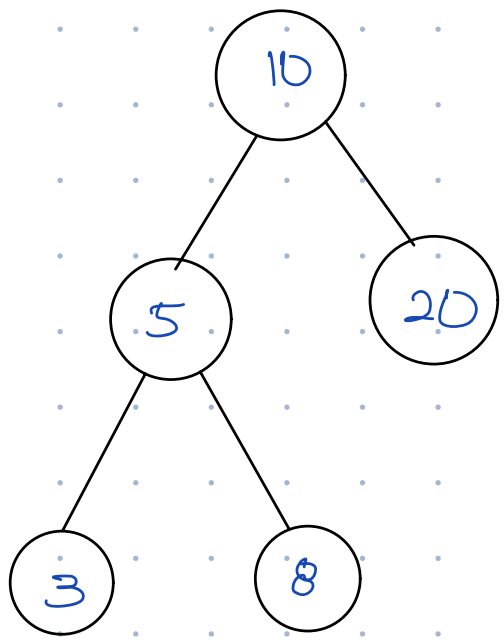


- All levels except possibly the last are filled
- Nodes are added in Right-to-left order

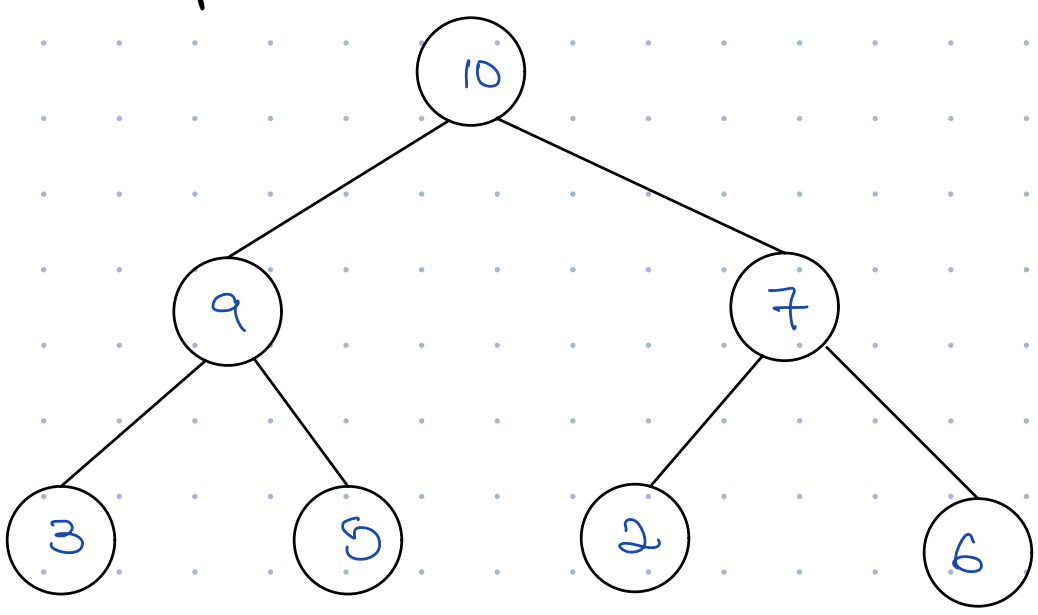
Array representation of a Binary Tree

- Parent(i) = $\lfloor i/2 \rfloor$
- Left(i) = $2i$
- Right(i) = $2i+1$

Example, [_ , 10 , 5 , 20 , 3 , 8]
1 2 3 4 5



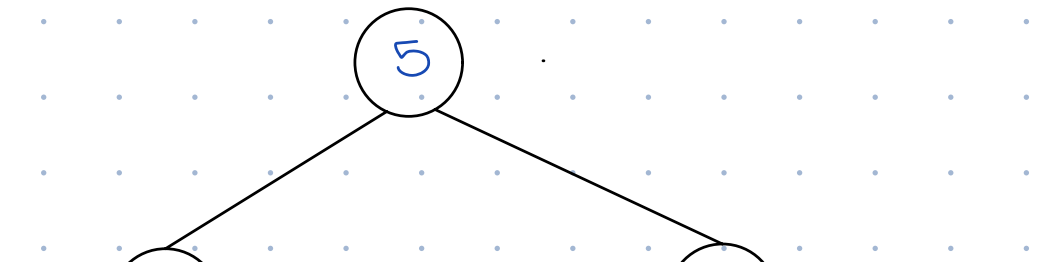
Max heap

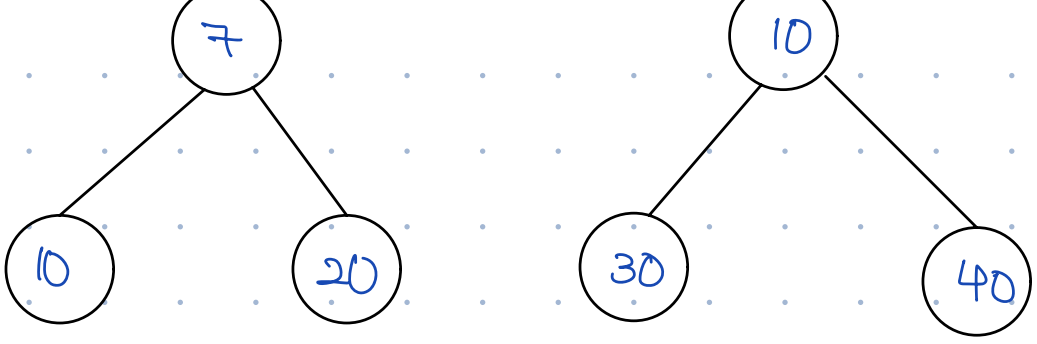


[_ , 50 , 30 , 20 , 15 , 10 , 8 , 6]

- All parent nodes ≥ it's child nodes

Min heap





[_, 5, 10, 20, 15, 30, 40]
1 2 3 4 5 6

- All parent nodes ≤ it's child nodes

NOTE:
- A tree that has only one node is both a max & min heap

Heapify Up/Insest: → Also called bottom up approach

Step 1: Insert new element at **end of array**

Step 2: Compare with parent → If larger, **swap**

Step 3: Repeat until parent is larger or root is reached

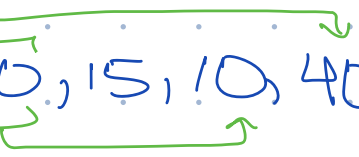
Example

Values to insert: 10, 20, 15, 30, 40

- [_, 10
- [_, 10, 20
  10 > 20 ×
  swap
  [_, 20, 10
- [_, 20, 10, 15
- [_, 20, 10, 15, 30
  10 > 30 ×
  swap
  [_, 20, 30, 15, 10
  20 > 30 ×
  swap

[_ , 30 , 20 , 15 , 10 ,

- [_ , 30 , 20 , 15 , 10 , 40



[_ , 30 , 40 , 15 , 10 , 20

[_ , 40 , 30 , 15 , 10 , 20]

Heapify Down / Delete: → Also called top-down approach

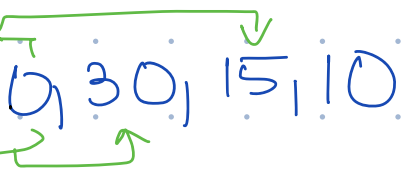
Delete always removes the **root (maximum element)**

- Step 1: Replace root with last node
- Step 2: Compare root with children, swap with larger child
- Step 3: Repeat until heap property restored

Example: [_ , 40 , 30 , 15 , 10 , 20]

- Delete():

[_ , 20 , 30 , 15 , 10



[_ , 30 , 20 , 15 , 10]

Practice:

[12, 7, 25, 4, 30, 8, 45, 2]

Insertion:

- [_ , 12

- [_ , 12, 7

- [_ , 12, 7, 25

12>25 ✗ swap

- [_ , 25, 7, 12

- [_ , 25, 7, 12, 4

- [_ , 25, 7, 12, 4, 30

7>30 ✗ swap

[_ , 25, 30, 12, 4, 7

25>30 ✗ swap

[_ , 30, 25, 12, 4, 7

- [_ , 30, 25, 12, 4, 7, 8

- [_ , 30, 25, 12, 4, 7, 8, 45

12>45 ✗ swap

[_ , 30, 25, 45, 4, 7, 8, 12

30 > 45 X
swap

[—, 45, 25, 30, 4, 7, 8, 12

- [—, 45, 25, 30, 4, 7, 8, 12, 2]

Delete:

[—, 45, 25, 30, 4, 7, 8, 12, 2]

NOTE:
Swap with largest child

- [—, 2, 25, 30, 4, 7, 8, 12]

2 > 30 X
swap

- [—, 30, 25, 2, 4, 7, 8, 12]

2 > 12 X
swap

[—, 30, 25, 12, 4, 7, 8, 2]
1 2 3 4 5 6 7

- [—, 2, 25, 12, 4, 7, 8]

[—, 25, 2, 12, 4, 7, 8]

2 > 7 X
swap

[—, 25, 7, 12, 4, 2, 8]